Programación Concurrente y Paralela

Práctica 6: Comunicación por Mensajes

2do Cuatrimestre 2026

Los ejercicios de esta práctica se pueden escribir en un pseudocódigo para comunicación por mensajes similar a Go pero simplificado:

Características generales

- Se pueden usar loops `while`.
- La asignación de variables es con `tipo nombre = valor`, sin el operador `:=` (siempre con el tipo).
- Para declarar tipos nuevos, pueden usar la sintaxis que prefieran, pueden ser `structs` como en Go, o declaraciones tipo clase, o estilo Haskell. Lo importante es que entendamos el tipo definido.
- Asumimos un valor **`nil`** tipado, el valor nulo del tipo. En particular **`nil channel`** es el canal nulo (para canales de cualquier tipo), donde enviar o recibir del canal nulo bloquea para siempre.
- Se tienen arrays de tamaño fijo (`int[k]`) y slices (`int[]`), que pueden crecer por medio de **`append`** y se pueden reducir haciendo slicing (`nombre[1:]`). También es válido escribirlos como `vector`, o `ArrayList`.
- Se puede usar **`range`** para números y también para arrays/slices.
- Iniciamos nuevos hilos de ejecución con la keyword `process`. Pueden estar definidos como funciones o ser anónimos (indicar qué valores reciben por parámetro).
- Todo pasaje de parámetros se entiende que es **por copia** (no hay memoria compartida). Las copias de canales apuntan a un mismo buffer del motor del lenguaje y se pueden usar para pasar datos.
- Se cuenta con las herramientas de Go basadas en tiempo como `Sleep`, `After` y `Ticker`.
- La impresión está simplificada en `print()`, toma de valores de consola en `input()`, si se quieren números aleatorios se puede usar `randInt(tope)`.
- Los canales son tipados, pero se cuenta con el tipo `any`, que permite hacer switch del **`type`** de un valor recibido por un canal.
- Usamos la siguiente sintaxis para canales:

  creación: `(channel tipo) nombre = create (channel tipo, k)` donde `k`≥ 0 es el tamaño del buffer del canal, 0 implica que el canal es sincrónico.

  envío-recepción: usamos `send(canal, valor)` y `receive(canal)`

  cierre: usamos `close(canal)`. Cuando se hace `receive` de un canal cerrado, se obtiene un objeto distinguido EOF que exclusivamente es indicador de un canal cerrado (no colisiona con otro valor posible de nada)

- Se cuenta con **`select`** y se puede usar la estrategia de cambiar canales por **`nil channel`**. Dentro del select se pueden usar los **`case`** o **`default`**, que no bloquea.
- En nuestro pseudocódigo **no** entendemos que todos los procesos lanzados desde main se terminan cuando el proceso principal termina. La idea es evitar tener que usar un `WaitGroup` para que `main` espere a todos los procesos. Los procesos que son servicios en un loop infinito pueden quedar esperando un nuevo mensaje por siempre; esa situación particular no es considerada como *deadlock*. Igual, necesitan los `WaitGroups` para otra cosa, los pueden usar, incluído el **`defer`** para hacer el `Done`.
- Dicho esto, en todos los ejercicios, procure que su solución no genere deadlocks y **justifique por qué no lo hace**.

Ejercicio 1. Se desea definir un servidor que utiliza un canal dado por parámetro para recibir dos tipos de mensajes: `Next` y `Count`. Cada vez que el servidor recibe un mensaje `Count` debe

mostrar por pantalla la cantidad de mensajes `Next` que recibió desde la última vez que recibió `Count`.

a) Definir una función que implementa el comportamiento del servidor (En Go, puede ser `func Srv(ch chan Cmd)` donde `Cmd` es un string que puede ser `"Next"` o `"Count"`).

b) Definir una función para el cliente, que tome un valor entero n, un canal para comunicarse con el servidor e indefinidamente envíe n mensajes `Next` seguido de un mensaje `Count` (En Go, sería `Client(n int, ch chan Cmd)`)

c) Escribir una función `main` que cree un sistema compuesto por un servidor y dos clientes que comuniquen a través de un mismo canal.

Ejercicio 2. Modificar la solución anterior para hacer que el servidor en lugar de mostrar por pantalla el número correspondiente, lo envíe al cliente y sea este último el que lo muestre por pantalla. Para ello, cada mensaje `Count` lleva adjunto un canal de respuesta por el cual el servidor devolverá el número (Cambie acordemente los tipos y signaturas que lo requieran)

Ejercicio 3. Se desea implementar un servidor que utiliza dos canales (pasados cómo parámetros) para comunicar con dos clientes. Cada cliente utiliza un canal para comunicar con el servidor números enteros. Cada cliente continuamente envía mensajes conteniendo números generados aleatoriamente (por ejemplo, con `rand.Intn`). El servidor continuamente espera recibir un par de mensajes (uno por cada cliente) y muestra por pantalla si los mismos coinciden o no.

(Implementado en Go, sería un servidor `Srv(ch1, ch2 chan int)`, un cliente que fuera `Cliente(ch chan int)` y la función `main()`, que componga el sistema)

Ejercicio 4. Se desea definir un servidor que provea el siguiente comportamiento: debe generar un número pseudoaleatorio entero en el rango [0,10]. El (en principio, único) cliente debe intentar adivinar el número que ha generado el servidor. Para ello envía sucesivamente mensajes conteniendo un número (puede usar cualquier estrategía para generar los números). Cada mensaje es contestado por el servidor indicando si el cliente acertó o no. El juego finaliza cuando el cliente acierta.

a) Dar una implementación para el servidor y para el cliente (En Go, funciones `Server` y `Cliente`, con los parámetros que considere necesarios), más una implementación de un sistema consistente en un servidor y un cliente.

b) Modificar su solución de manera tal que varios clientes puedan usar secuencialmente al servidor (Un cliente comienza a jugar cuando el anterior ha finalizado)

c) Dar una solución que permita a varios clientes jugar concurrentemente (cada uno debe adivinar su propio número).

Ejercicio 5. Para este ejercicio, suponga que sólo cuenta con canales **sincrónicos** e implemente un canal **asincrónico** de capacidad (virtualmente) ilimitada, usando un canal de entrada y uno de salida. (En Go, implemente una función `AsyncChan` que use los canales `entrada` y `salida` de tipo `chan any` (`any` es lo mismo que `interface{}`))

Ejercicio 6. ¿Cómo modificaría la solución del ejercicio 4 si el modelo de comunicación es asincrónico y el cliente puede enviar mensajes aún cuando el servidor no ha respondido a los mensajes anteriores?

Ejercicio 7. Se desea definir un servidor que funcione como una variable compartida a la que los clientes pueden acceder mediante oportunos mensajes.

a) Dar una solución considerando comunicación sincrónica.

b) ¿Es correcta su solución si la comunicacación es asincrónica? En caso contrario, modificar la solución anterior.

Ejercicio 8. Se desea resolver el siguiente problema utilizando comunicación por intercambio asincrónico de mensajes

a) Se desea modelar un proceso Timer que recibe al momento de su creación un valor indicando la frecuencia entera en milisegundos con la que deberá generar ticks y la lista de canales en los que deberá señalar tales ticks (en Go, un slice de canales, por ejemplo `chans := []chan struct{}{ch1, ch2, ch3, ch4}`). Crear un proceso timer que cada `t` milisegundos genere mensajes de tipo unit (en Go, `struct{}{}`) en los canales `ch1`, `ch2`, `ch3` y `ch4`.

b) Modificar la solución anterior asumiendo que el Timer se crea inicialmente con una lista vacía de canales y que los clientes que necesiten recibir los ticks deberán registrarse primero. Para ello se comunicarán con Timer a través de un canal dado al momento de la creación del timer. Los pedidos de registración pueden arribar en cualquier momento.

Ejercicio 9. Se desea definir un servicio de codificación y transmisión de información llamado T. El servicio T recibe un string que un cliente C quiere enviar a un servicio remoto R. El servicio T es el encargado de codificar cada mensaje recibido desde C y de reenviarlo a R. Puede asumir que cuenta con un función `codificar` que toma un string `m` y devuelve la codificación de `m`.

Se solicita

a) Dar el código para el servicio T.

b) Suponer ahora que el servicio T debe permitir a un cliente reenvíar la información a distintos servicios remotos. Para ello el cliente debe tener la posibilidad de enviar servicio T un aviso de quién será el destinatario de los mensajes que se enviarán a continuación (los avisos de cambio de destinatario deben ser mensajes individuales distintos de los textos a codificar). Por ejemplo, el cliente puede comenzar indicando al servicio T que los mensajes que se enviarán a continuación deberán ser reenviados al servicio remoto $R1$, luego todos los mensajes que envíe a continuación el cliente, y hasta tanto este no notifique a T que desea cambiar el destinatario, deberán ser codificados y reenviados al servicio remoto $R1$. Asumir que el cliente siempre comenzará indicando al servicio T cual es el primer servicio remoto al que se deberán reenviar los mensajes. Sin embargo, no puede realizar ninguna suposición sobre la cantidad de mensajes que se enviarán a cada servidor remoto. Dar una solución para el servicio T que exhiba el comportamiento descripto.

c) Suponer ahora que la función `codificar` toma, además del texto a codificar, una clave (otro `string`) que es generada por otro servicio de codificación K. Esta clave debe ser

solicitada enviando un canal propio (para recibirla) a un canal que T recibe por parámetro. Cada mensaje lleva una **clave distinta** que debe ser oportunamente pedida (intentando minimizar las esperas pero sin bloquear innecesariamente a K). Dar una implementación modificada para el servicio T.

Ejercicio 10. Se cuenta con un servicio S que permite hacer cálculos numéricos computacionalmente costosos. S recibe cada pedido de cálculo a realizar en un mensaje sobre el canal dado a momento de creación del servicio. El mismo es procesado y la respuesta es enviada al cliente a través de un canal indicado por el cliente al momento de realizar la solicitud. Es decir, un cliente C envía un mensaje con su input (un entero) y el canal donde espera recibir la respuesta sobre el canal en que S atiende requerimientos. El cliente fue implementado de manera tal que puede enviar varios pedidos de cálculo sin esperar a que los anteriores hayan sido procesados. Por otro lado S no puede garantizar el correcto funcionamiento cuando recibe un nuevo mensaje y aún está procesando un mensaje anterior. Por este motivo, se desea implementar otro servicio proxy P que intermedie la interacción entre C y S. La idea es que el cliente C no interactuará directamente con S y en su lugar interactuará con P. En este escenario P reenvía a S todos los mensajes que recibe de C y viceversa, pero asegurando que S nunca recibirá un nuevo mensaje hasta que no haya concluido de procesar el pedido anterior.

a) Dar una implementación para P que recibe los canales propio y el de S, en ambos se recibirán estructuras `Req` que consisten de un entero y el canal de enteros para las respuestas.

b) Suponer ahora que S puede manejar a lo sumo N mensajes concurrentes de manera correcta. Extender la solución propuesta anteriormente para P de manera tal de maximizar la concurrencia. Asumir que S produce los resultados respetando el orden en que llegan los pedidos siempre y cuando haya a lo sumo N pedidos concurrentes.

c) Extender la implementación de P para aprovechar el hecho de que existen dos copias de S (que comunican por canales distintos). P debe aprovechar las dos copias de S a fin de maximizar la concurrencia. Debe considerar que al cliente se le deben retornar los resultados en el orden en que hizo los requerimientos.